Software Architecture and Design Specification

Project: FAQ Builder

Version: 1.0

Authors: Example Team

Date: 15-09-2025

Status: Draft

Revision History

Version | Date | Author | Change Summary

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the **FAQ Builder System**. It translates the requirements defined in the SRS and the validation plan in the STP into an architectural blueprint. The primary goal is to provide developers, testers, system administrators, and stakeholders with a comprehensive view of how the FAQ Builder system will be structured, integrated, and maintained.

1.2 Scope

The FAQ Builder system provides an **Admin Dashboard** to create, update, delete, and manage Question–Answer (Q/A) pairs, and an **End-User Search Interface** that allows users to search and retrieve relevant answers.

- Core functionalities: CRUD operations for Q/A pairs, tagging, publishing workflow, keyword-based search with ranking, suggested answers, and analytics.

- Non-functional requirements: secure communications (TLS 1.2+), response times ≤ 2 seconds for 95% of queries, role-based authentication, 99% availability, audit logging, and mobile responsiveness.

- Out of scope: AI-based semantic search, third-party knowledge base integrations, advanced internationalization.

1.3 Audience

This document is intended for:

- **Developers**: to implement the modules and APIs.
- **QA/Test Engineers**: to design test cases and validate implementation.
- **System Administrators/DevOps**: to manage deployment, monitoring, and backups.
- **Content Managers & Admins**: to understand workflows.
- **Security Auditors**: to validate compliance and risk controls.

1.4 Definitions
CRUD – Create, Read, Update, Delete

RBAC – Role-Based Access Control

JWT – JSON Web Token

UI – User Interface

API – Application Programming Interface

CTR – Click-Through Rate

RTM – Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document
Provides architectural deliverables including UML diagrams, ADRs, threat models, and API design.

2.2 Related Documents
SRS, STP, RTM.

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide a secure, scalable FAQ management system.
- Ensure fast search (≤ 2s for 95% of queries).
- Maintain 99% availability with fault tolerance.
- Support role-based access and audit logging.

Constraints:

- Must comply with TLS 1.2+ encryption standards.
- PCI-DSS and GDPR considerations for audit and data handling.
- Limited scope: no AI semantic search or third-party KB integrations.

3.2 Stakeholders & Concerns

End Users: Expect fast, relevant, and accurate answers.

Admins/Editors: Need intuitive CRUD workflow, tagging, and publishing.

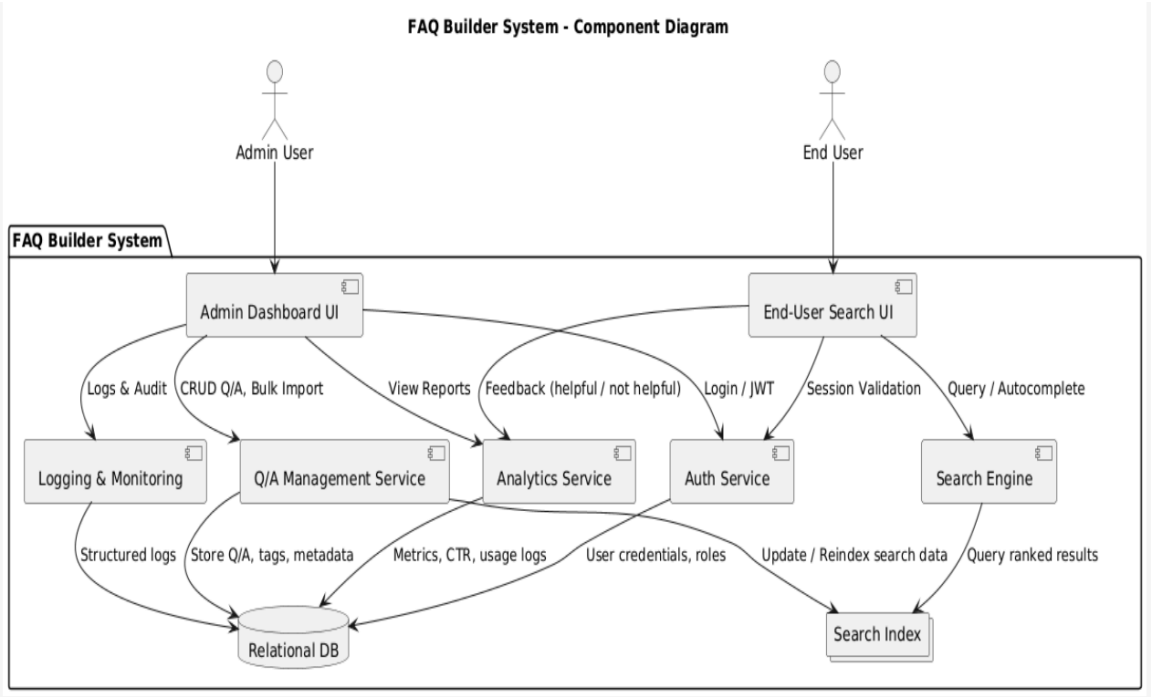
QA/Testers: Need APIs and workflows traceable to requirements/test cases.

System Admins/DevOps: Focus on deployment, monitoring, and backups.

Regulators/Auditors: Require compliance, logging, and data protection.

Developers: Prefer modular, maintainable architecture with clear APIs.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Admin Dashboard: Web UI for managing content.

Search UI: User-facing interface with responsive design.

Auth Service: Validates credentials, issues JWT, enforces RBAC.

Q/A Management API: Manages lifecycle of Q/A entries.

Search Engine Integration: Provides ranked search results and reindexing.

Analytics Module: Collects performance metrics and user feedback.

Database: Relational DB for structured data and logs.

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered architecture with REST APIs and search engine integration.

Rationale:

- Provides clear separation of concerns.
- Easier testing (unit, integration, system) as per STP.
- Microservices not needed for current scale but possible future evolution.

3.6 Technology Stack & Data Stores

Frontend: React or Angular (Admin UI & Search UI).

Backend: Java Spring Boot / Node.js REST APIs.

Database: MySQL/PostgreSQL for Q/A, tags, users.

Search Engine: Elasticsearch/Meilisearch.

Security: TLS 1.2+, JWT, RBAC.

Monitoring: Prometheus, Grafana, structured JSON logs.

3.7 Risks & Mitigations

Risk: Search engine downtime → Mitigation: Fallback cached responses.

Risk: Unauthorized access → Mitigation: RBAC + JWT + rate limiting.

Risk: Performance degradation under load → Mitigation: Load testing + indexing optimizations.

Risk: Data corruption → Mitigation: Automated backups + restores.

3.8 Traceability to Requirements

QA-F-010 → Q/A Management API

QA-F-020 → Search Engine Integration

QA-F-030 → Feedback API

FAQ-NF-001 → Response times validated by monitoring/performance tests

3.9 Security Architecture

STRIDE Threat Modeling:

- **Spoofing:** JWT-based authentication with secure login.
- **Tampering:** Audit logs with write-once storage.
- **Information Disclosure:** TLS 1.2+, encrypted data at rest.
- **DoS:** API rate limiting + monitoring.
- **Elevation of Privilege:** Strict RBAC enforcement.

Validation: Covered by security testing (XSS, SQLi, brute force, penetration)

4. Design

4.1 Design Overview

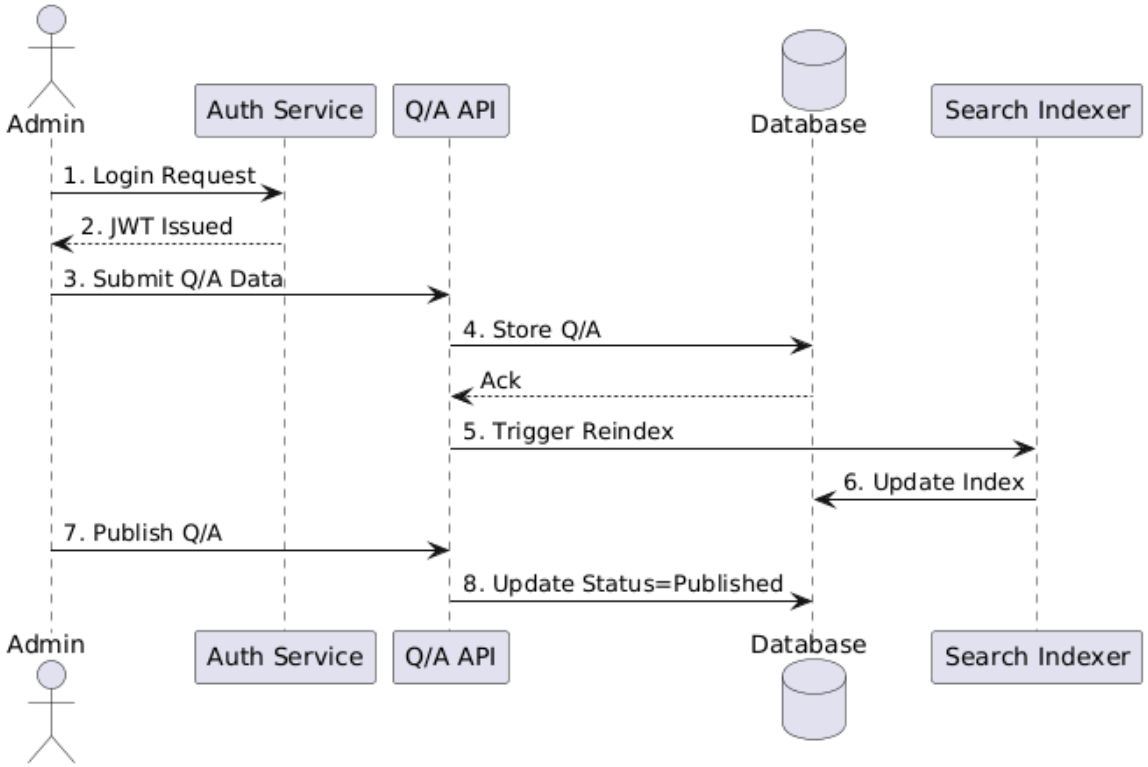
The FAQ Builder system is designed using a **layered architecture** to ensure modularity, maintainability, and scalability:

- **Presentation Layer**:
    - *Admin Dashboard* (for CRUD, tagging, publishing, analytics)
    - *End-user Search UI* (for search, filters, suggestions, feedback)
- **Application Layer**:
    - REST APIs for authentication, Q/A management, search, feedback, analytics, and monitoring.
- **Integration Layer**:
    - Connection to the indexing/search engine (Lucene/Elasticsearch/Meilisearch).
- **Data Layer**:
    - Relational Database for Q/A pairs, tags, users, and logs.
    - Search Index for keyword ranking and highlighting.

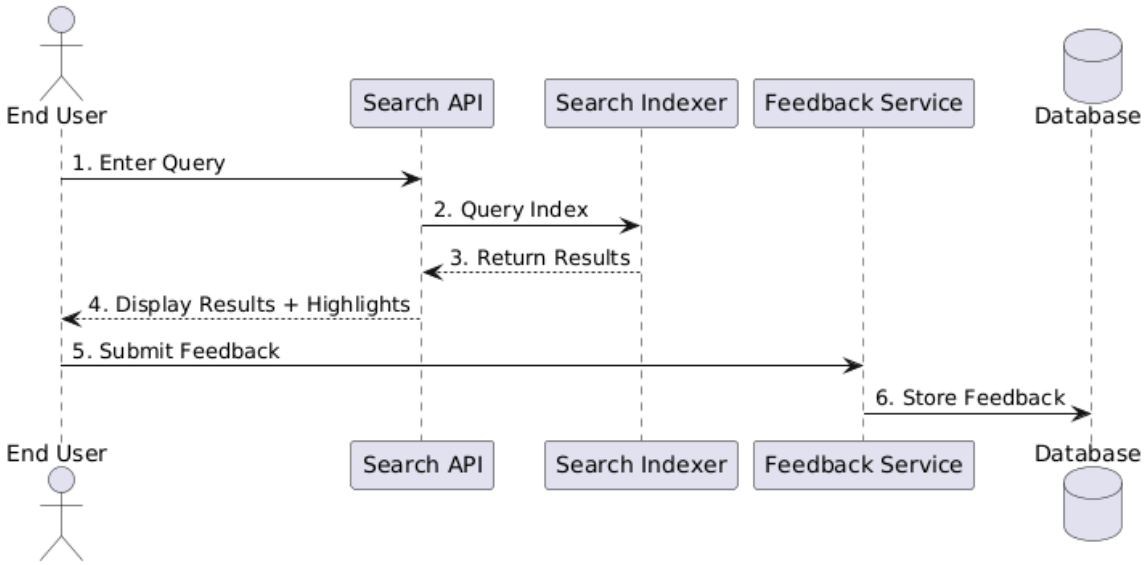
This separation of concerns allows easier updates, role-based permissions, and compliance with the performance and security requirements

4.2 UML Sequence Diagrams

Admin Creates & Publishes a Q/A Pair:



End User Searches for Answer:



4.3 API Design

Auth API:

Endpoint: /api/auth/login

Method: POST

Request: { username, password }

Response: { jwtToken, role }

Errors: 401 Invalid Credentials, 403 Account Locked

Q/A Management API

Endpoint: /api/qa

Method: POST

Request: { question, answer, tags[], status, language }

Response: { id, status, createdAt }

Errors: 400 Validation Error, 401 Unauthorized

Endpoint: /api/qa/{id}

Method: PUT

Request: { question, answer, tags[], status }

Response: { id, status, updatedAt, version }

Errors: 400 Validation Error, 401 Unauthorized, 404 Not Found

Endpoint: /api/qa/{id}

Method: DELETE

Request: { }

Response: { id, status: archived }

Errors: 401 Unauthorized, 404 Not Found

Bulk Import API:

Endpoint: /api/qa/bulk-import

Method: POST

Request: { file: CSV/JSON }

Response: { successCount, errorCount, errors[] }

Errors: 400 Invalid File Format, 401 Unauthorized

Search API:

Endpoint: /api/search

Method: GET

Request: { q: string, filters: {tag, date, language}, sort, page }

Response: { results: [{id, question, snippet, tags[]}], total }

Errors: 400 Invalid Query, 500 Index Error

Endpoint: /api/search/reindex

Method: POST

Request: { mode: "full" | "partial" }

Response: { status: "reindex_started" }

Errors: 401 Unauthorized, 500 Index Error

Feedback API:

Endpoint: /api/feedback

Method: POST

Request: { qaId, helpful: true|false, userId/sessionId }

Response: { status: "recorded" }

Errors: 400 Validation Error, 500 Server Error

4.4 Error Handling, Logging & Monitoring

Error Handling: Standard HTTP error codes (401 Unauthorized, 403 Forbidden, 422 Validation Error).

Logging: Structured JSON logs; no sensitive data stored; each log includes timestamp and request ID.

Monitoring: Metrics include transaction error rate, search latency, failed logins, indexing delays, uptime.

4.5 UX Design

Admin Dashboard: Q/A table view, create/edit forms, workflow states (draft → review → published), analytics cards.

End-user Search UI: Search box with autocomplete, filters, keyword highlighting, pagination, related queries, and feedback buttons.

Accessibility: WCAG 2.1 AA compliance, ARIA roles, keyboard navigation, high-contrast mode.

Responsive Design: Supports desktops, tablets, and mobile devices.

4.6 Open Issues & Next Steps

Support for **multi-language stemming and search ranking**.

Integration with **external analytics/reporting tools**.

Potential **chatbot integration** for conversational FAQ queries.

Explore **learning-to-rank** algorithms to improve search relevance based on user behavior.

5. Appendices

- **CRUD** – Create, Read, Update, Delete (operations for managing Q/A pairs)
- **RBAC** – Role-Based Access Control (managing user roles & permissions)
- **JWT** – JSON Web Token (authentication token used for secure API access)
- **UI** – User Interface (Admin Dashboard and End-User Search interface)
- **API** – Application Programming Interface (REST APIs for Q/A management, search, analytics)
- **CTR** – Click-Through Rate (analytics metric to measure engagement with FAQ results)
- **RTM** – Requirements Traceability Matrix (mapping between requirements, design, and test cases)

5.2 References

- IEEE 42010: Recommended practice for architecture description
- OWASP Application Security Verification Standard (ASVS)
- NIST SP 800-160: Systems Security Engineering
- Software Requirements Specification (SRS) – FAQ Builder v1.0

- Software Test Plan (STP) – FAQ Builder v1.0
  Copy of Test_Plan_Template for …
- Requirements Traceability Matrix (RTM) – FAQ Builder v1.0

5.3 Tools

- **Diagramming:** PlantUML, draw.io (for UML & architecture diagrams)
- **API Design:** Swagger / OpenAPI
- **Testing Tools:** Selenium, Postman, JMeter, REST Assured, OWASP ZAP
  Copy of Test_Plan_Template for …
- **Version Control & CI/CD:** GitHub/GitLab, Jenkins/GitHub Actions
- **Monitoring:** Prometheus, Grafana, ELK stack